Searching a Large Student Record Database: Design, Analysis and Validation

Course: Design and Analysis of Algorithms, Assignment 1 (CO1)
Topic: Efficient search strategy for a university student record system

1. Summary

I compared three ways of finding a student by registration number: **linear search**, **binary search** on sorted records, and a **hash table**. For each one I derived the cost mathematically, then implemented it and measured it on 1,000, 10,000, 100,000 and 1,000,000 records. The main findings are:

- **Linear search does not scale.** Going from 1,000 to 1,000,000 records made it about 1,000 times slower (44.5 µs to 55,103 µs per search). At a million records it needs roughly 730,000 comparisons per search.
- **Binary search scales very well.** The same thousandfold growth only raised the comparisons from 9.4 to 19.5, which matches the log n prediction. At a million records it was about 7,000 times faster than linear search.
- **Hashing is fastest per search** (about 1 comparison and 1.3 µs at a million records), but it needs the most memory and has a poor worst case if many keys collide.
- **Preprocessing pays for itself quickly.** Sorting cost 4.5 s and building the hash table 2.3 s at a million records. That is repaid after roughly 82 and 42 searches respectively.
- **Recommendation.** For the original problem I recommend binary search on a sorted array. For the Day-4 scenario (50,000 searches per hour, one update per day) I recommend **modifying** the design by adding a hash index on top of the sorted array, rebuilt once a day.

2. Introduction

A university stores each student record under a unique registration number. Staff need to check whether a record exists and fetch it. The database could hold anywhere from 1,000 to over 1,000,000 records, so a method that feels instant on a small database may become unusable on a large one.

This report covers the whole assignment. It designs three different algorithms (Section 3), analyses them mathematically (Section 4), tests them experimentally (Sections 5 and 6), compares theory with practice (Section 7), recommends a design (Section 8), and re-evaluates that design for the changed Day-4 requirement (Section 9). Any preprocessing, such as sorting, is counted in the cost throughout. A fast search that needs an expensive setup is not really fast unless the setup is repaid.

Notation. n is the number of records, q is the number of searches, m is the hash table size, and $\alpha = n/m$ is the hash load factor. All logarithms are base 2.

3. The Three Approaches

I chose three algorithms built on three different ideas, so that the comparison is meaningful and not just three versions of the same thing.

	A. Linear search	**B. Binary search**	**C. Hash lookup**
Style	Iterative	Recursive	Iterative (table build and lookup)
Design strategy	Brute force	Decrease and conquer	Transform and conquer (space-time trade-off)
Preprocessing	None	Sort by registration number	Build a hash table
Core idea	Check records one by one	Halve the search range each step	Compute where the record must be

3.1 Linear search

Start at the first record and compare its registration number with the key, then move on until the key is found or the records run out. It needs no preprocessing and works on unsorted data. I included it as the natural baseline: for one or two lookups it is hard to beat because it has no setup cost.

```
LinearSearch(A[0..n-1], key)
    for i <- 0 to n-1 do
```

```
    if A[i].regNo = key then return i
  return -1
```

3.2 Binary search

Records are kept sorted by registration number. The algorithm compares the key with the middle record. If they match it stops. Otherwise it throws away the half that cannot contain the key and repeats on the other half. The sort is done once with merge sort (about n log n comparisons). I chose this to show how a one-time sorting cost buys much faster searches. The recursive form also gives a clean recurrence for Sections 4.6 and 4.7. An iterative version of binary search would be the same algorithm, so I do not count it as a separate approach.

```
BinarySearch(A, low, high, key)
  if low > high then return -1
  mid <- floor((low + high) / 2)
  if A[mid].regNo = key then return mid
  else if key < A[mid].regNo then return BinarySearch(A, low, mid-1, key)
  else return BinarySearch(A, mid+1, high, key)
```

3.3 Hash table lookup

Instead of narrowing a range, this approach calculates where the record should be. The hash function is h(key) = key mod m, where m is a prime close to n / 0.75, so the load factor is about 0.75. Keys that land in the same bucket are stored in a short chain. I chose this because it works on a different principle from the other two and trades memory for speed.

```
BuildHash(A[0..n-1], m)
  create table T[0..m-1] of empty lists
  for i <- 0 to n-1 do
    append (A[i].regNo, i) to T[A[i].regNo mod m]


HashSearch(T, m, key)
  for each entry e in T[key mod m] do
```

```
        if e.key = key then return e.index
    return -1
```

4. Theoretical Analysis

4.1 Basic operation

Algorithm	Basic operation	Where it occurs	Why it is the right one to count
Linear	Comparison A[i] = key	Once per loop pass	It is the most repeated step. Everything else in the loop happens the same number of times.
Binary	Comparison of key with A[mid]	Once per recursive call	Each call does exactly one, so comparisons equal calls.
Hash	Key comparison while scanning a chain	Inside the bucket scan	Computing key mod m costs the same for every n, so only chain scanning varies.

As n grows, the comparison count grows in step with n for linear search, by one per doubling for binary search, and stays roughly constant for hashing (as long as m grows with n).

4.2 Deriving the time-complexity functions

The input size is n, the number of records.

Linear search. If the key sits at position i, the search makes i comparisons. The best case is 1. The worst case is n (key last, or not present). If a present key is equally likely to be at any position:

$C_{avg}(n) = (1/n)(1 + 2 + ... + n) = (1/n) \cdot n(n+1)/2 = (n+1)/2$

If the key is present with probability p, then $C_{avg} = p(n+1)/2 + n(1-p)$. There is no preprocessing, so the cost is $\Theta(n)$ per search.

Binary search. After the first comparison at most $\lfloor n/2 \rfloor$ records remain, and after k comparisons at most $\lfloor n/2^k \rfloor$. The range is empty once $n/2^k < 1$, that is, when $k > \log n$. So the worst case is $\lfloor \log n \rfloor + 1$ comparisons (proved formally in Section 4.7).

For the average successful search, take n = 2^k − 1. Exactly 2^(i−1) keys are found at comparison number i, so the total over all keys is

$\Sigma\, i \cdot 2^{(i-1)} = (k-1) \cdot 2^k + 1$

and the average is about k − 1, which is roughly log n − 1. Almost all keys sit near the bottom of the search tree, which is why the average is only one comparison below the worst case.

The preprocessing sort satisfies M(n) = 2M(n/2) + n, which gives about n log n comparisons.

Hash lookup. Assume the hash function spreads keys evenly over the m buckets, so the expected chain length is $\alpha = n/m$.

- An unsuccessful search scans a whole chain: about α comparisons.
- A successful search makes one comparison for the key itself plus one for each key placed earlier in its bucket. The i-th inserted key has (i−1)/m such keys on average, so the average is $1 + (1/n)\,\Sigma\,(i-1)/m = 1 + (n-1)/(2m) \approx 1 + \alpha/2$. With α = 0.75 this is about 1.4.
- Building the table takes n insertions at constant expected cost, so Θ(n).

4.3 Preprocessing and searching together

For q searches, the total cost is the preprocessing cost plus q times the cost per search.

Approach	Preprocessing	Average per search	Total for q searches
Linear	none	(n+1)/2	Θ(qn)
Binary	about n log n	about log n − 1	Θ(n log n + q log n)
Hash	about n	about 1 + α/2	Θ(n + q) expected

For n = 1,000,000, linear search averages about 500,000 comparisons per successful search. Binary search with sorting costs about 2.0 × 10^7 for the sort plus about 19 per search, so it becomes cheaper once q is above roughly 40. Hashing costs about 10^6 to build plus about 1.4 per search, so it wins after only a handful of searches. In operation counts, preprocessing is repaid very quickly. Section 7 shows how the real timings changed this picture.

4.4 Best, average and worst cases

Linear search. The best case is the key being the very first record (1 comparison). The worst case is the key being last or absent, which forces n comparisons, since every record must be ruled out. The average case, with the key equally likely anywhere, is (n+1)/2. The spread exists because the algorithm has no information about where the key is, so position alone decides how far it scans.

Binary search. The best case is the key sitting exactly at the middle of the sorted array (1 comparison). The worst case is a missing key or one at the deepest level of the search tree, taking ⌊log n⌋ + 1 comparisons. The average successful search is about log n − 1. Half of all records sit at the deepest level of the tree and a quarter at the level above, so most keys need nearly the maximum number of steps.

Hash lookup. The best case is 1 comparison (or 0 for a missing key whose bucket is empty). The average is about 1 + α/2. The worst case is n comparisons, which happens when every key hashes to the same bucket and the chain degenerates into a linear scan. This needs a poor hash function or very patterned keys. Using a prime table size makes that unlikely, and Section 6.3 demonstrates it on purpose.

4.5 Asymptotic analysis

Approach	Best	Average	Worst	Preprocessing
Linear	Θ(1)	Θ(n)	Θ(n)	none
Binary	Θ(1)	Θ(log n)	Θ(log n)	Θ(n log n)
Hash	Θ(1)	Θ(1) expected	Θ(n)	Θ(n)

In order of growth, O(1) < O(log n) < O(n) < O(n log n). Here is what that means for the university as the database grows:

n	Linear (average, successful)	Binary (worst)	Hash (average)	Sort cost (n log n)
1,000	about 500	10	about 1.4	about 1.0 × 10^4
10,000	about 5,000	14	about 1.4	about 1.3 × 10^5
100,000	about 50,000	17	about 1.4	about 1.7 × 10^6

1,000,000	about 500,000	20	about 1.4	about 2.0 × 10^7

Growing the database a thousandfold makes linear search a thousand times slower, binary search only twice as slow (10 to 20 comparisons), and hashing no slower at all. Linear search is fine for a small department database but stops being practical well before a million records.

4.6 Recurrence relation for the recursive approach

Take the recursive binary search and let n = high − low + 1 be the size of the current range.

- **Original problem size:** n records.
- **Subproblem size:** after comparing with A[mid], the algorithm recurses on one side only. The left side has ⌊(n−1)/2⌋ records and the right side ⌊n/2⌋. The worst case takes the larger side, ⌊n/2⌋.
- **Number of recursive calls:** exactly 1, because the algorithm never searches both halves.
- **Work outside the recursive call:** one comparison with A[mid] plus constant arithmetic, so 1 basic operation.
- **Base case:** an empty range returns −1 with no comparison. A one-record range needs one comparison, so T(1) = 1.

Putting these together:

T(n) = T(⌊n/2⌋) + 1 for n > 1, with T(1) = 1

For the merge sort used in preprocessing, the sort splits into two halves and merges them in linear time, so M(n) = 2M(n/2) + n with M(1) = 0.

4.7 Solving the recurrence

I used the iteration method (backward substitution), then confirmed it by induction and with the Master Theorem. The method applies because each step shrinks the problem by a constant factor and adds a constant amount of work, so the recurrence unrolls cleanly when n = 2^k.

Iteration for n = 2^k:

T(2^k) = T(2^(k−1)) + 1 = T(2^(k−2)) + 2 = ... = T(2^(k−i)) + i

Setting i = k gives T(2^k) = T(1) + k = 1 + k = log n + 1.

Induction for any n. Claim: T(n) = ⌊log n⌋ + 1. The base case n = 1 gives 1, which is correct. For n ≥ 2, assuming the claim for smaller values:

T(n) = T(⌊n/2⌋) + 1 = (⌊log ⌊n/2⌋⌋ + 1) + 1 = (⌊log n⌋ − 1) + 2 = ⌊log n⌋ + 1

Master Theorem check. The recurrence has the form T(n) = aT(n/b) + f(n) with a = 1, b = 2 and f(n) = Θ(n^0). Since a = b^0 (1 = 2^0), this is the case T(n) = Θ(n^0 log n) = **Θ(log n)**. For merge sort, a = 2, b = 2 and d = 1, so again a = b^d and M(n) = Θ(n log n).

What it means. For 1,000,000 records at most 20 comparisons are needed. Each doubling of the database adds one more comparison. A database of one billion records would still need only 30. Because the recursion depth is the same as the comparison count, the recursive version never goes more than about 20 calls deep here, so stack space is not a concern.

5. Experimental Setup

5.1 What was built

All three algorithms and the test harness are in Python_Code.py (Python, plus matplotlib for the graphs). Each search returns both the position and the number of comparisons, so the experiments measure the same basic operation that the theory counts, as well as time.

5.2 Test data and procedure

- **Dataset sizes:** 1,000, 10,000, 100,000 and 1,000,000 records.
- **Registration numbers:** unique random 10-digit integers, generated with a fixed random seed (42) so the run can be repeated exactly.
- **Same data for every algorithm:** at each size, linear search and hashing use the same unsorted list, and binary search uses a sorted copy of it.
- **Queries:** 100 per dataset size, namely 50 registration numbers that exist and 50 that do not. The same 100 queries are given to all three algorithms.
- **Measurements:** average comparisons per search, average time per search (using time.perf_counter), and preprocessing time for the sort and the hash build. Merge sort comparisons were also counted.
- **Preprocessing** is timed separately from searching, so it can be added back in the way Section 4.3 describes.

5.3 Correctness testing

Correctness was checked in two ways.

1. **Edge cases:** empty list, single record, first record, last record, a key smaller than every record, a key larger than every record, and a key that falls between two records. All three algorithms gave the expected answer, and merge sort sorted a small unsorted list correctly.
2. **Random verification at every size:** for each of the 100 queries, a present key had to return a position that actually holds that key, and a missing key had to return −1. All three algorithms passed at all four sizes.

5.4 Machine

(Fill in your own details here: processor, RAM, operating system, Python version. The numbers below are from one run and will differ a little on another machine.)

6. Results

(Insert Output.png here as Figure 1. The top row shows comparisons and time per search, the bottom left shows preprocessing time, and the bottom right shows the Day-4 daily cost.)

6.1 Searching

Average comparisons per search

n	Linear	Binary	Hash
1,000	748.9	9.4	1.07
10,000	7,652.5	12.8	1.17
100,000	77,820.8	16.3	1.13
1,000,000	729,212.5	19.5	0.99

Average time per search (microseconds)

n	Linear	Binary	Hash
1,000	44.5	2.17	0.52
10,000	449.5	3.34	0.68
100,000	5,283.9	5.61	0.96
1,000,000	55,103.5	7.89	1.29

6.2 Preprocessing

n	Merge sort (s)	Merge sort comparisons	Hash build (s)
1,000	0.002	8,707	0.000
10,000	0.023	120,350	0.004
100,000	0.408	1,536,422	0.135
1,000,000	4.513	18,673,721	2.302

6.3 Best and worst cases (n = 100,000)

Case	Comparisons
Linear, key is the first record	1
Linear, key is the last record	100,000
Linear, key is missing	100,000
Binary, key is the middle record	1
Binary, key is missing (smaller than every record)	16
Hash, random present keys (average of 1,000)	1.34
Hash, deliberately colliding keys (n = 5,000)	5,000

For the last row, I built 5,000 keys that were all multiples of the table size, so every key landed in one bucket. Searching for the last one then took 5,000 comparisons, which is exactly the linear-search worst case. This confirms the weakness described in Section 4.4.

7. Comparing Theory with the Experiments

7.1 Comparisons

The workload is half present keys and half missing keys, so the theoretical averages have to be adjusted to match it.

n	Linear: expected 0.75n	Linear: measured	Binary: expected range	Binary: measured	Hash: expected	Hash: measured
1,000	750	748.9	9.0 to 10	9.4	about 1.06	1.07
10,000	7,500	7,652.5	12.3 to 14	12.8	about 1.06	1.17
100,000	75,000	77,820.8	15.6 to 17	16.3	about 1.06	1.13
1,000,000	750,000	729,212.5	18.9 to 20	19.5	about 1.06	0.99

Where these expectations come from:

- **Linear:** present keys average n/2 and missing keys cost n, so a 50/50 mix averages 0.75n. This is why my measured numbers are higher than the (n+1)/2 figure for successful searches alone.
- **Binary:** the lower end is the successful-search average ($\log n - 1$) and the upper end is the worst case ($\lfloor \log n \rfloor + 1$). The measurements fall inside the range every time.
- **Hash:** successful searches cost about $1 + \alpha/2 \approx 1.375$ and missing keys about $\alpha \approx 0.75$, giving roughly 1.06 for the mix. Missing keys that fall in an empty bucket cost 0, which also pulls the average down. When tested on present keys only, the average was 1.34, close to the predicted 1.4.

The small differences (for example linear at 1,000,000 records is about 3% below the prediction) come from using only 50 random present keys per size, so the sample average wanders a little from the true average.

7.2 Growth

When n grew by a factor of 1,000, the measured comparisons grew as follows:

	Predicted growth	**Measured growth**
Linear	about 1,000 times	about 974 times
Binary	about 2 times (10 to 20)	about 2.1 times (9.4 to 19.5)
Hash	about 1 (flat)	about 0.9 (flat)

So the comparison counts agree with $\Theta(n)$, $\Theta(\log n)$ and $\Theta(1)$ very closely. Linear search's time also grew almost exactly tenfold for each tenfold increase in n (44.5, 449.5, 5,283.9, 55,103.5 μs).

The merge sort comparisons also match theory. At 1,000,000 records the measured 18.7 million is close to the n log n estimate of about 20 million, and just under the exact worst-case bound of $n \log n - n + 1 \approx 18.9$ million.

7.3 Where practice differed from theory, and why

- **Time grew faster than comparisons for binary search and hashing.** Binary search comparisons only doubled over the 1,000-fold growth, but its time grew about 3.6 times (2.17 to 7.89 μs). Hash comparisons stayed flat but time still rose about 2.5 times (0.52 to 1.29 μs). The likely reason is memory. A bigger dataset no longer fits in the processor's fast cache, so each lookup is more likely to wait on slower memory. The theory counts comparisons and ignores memory speed. I did not measure cache behaviour directly, so this is an explanation, not a proven cause.
- **The hash build did not scale linearly.** Theory says $\Theta(n)$, which predicts a tenfold rise per step. The measured build time rose about 34 times from 10,000 to 100,000 records, and about 17 times from 100,000 to 1,000,000. Building the table allocates over a million small lists in Python, and memory allocation and cache effects add overhead that the simple operation count ignores.

- **Break-even points were higher than predicted.** The operation-count model suggested about 40 searches for binary search and about 2 for hashing. The measured times give about 82 and 42. This is because the hash build is relatively expensive in Python compared with one linear scan, as described above.
- **The gap between binary search and hashing was smaller than the operation counts suggest.** Hashing needs far fewer comparisons, but per day it was about 3.6 times cheaper than binary search (3.85 s against 13.98 s, Section 9), not the roughly 16 times the operation counts implied.
- **A worst-case detail.** For 100,000 records the worst-case bound is $\lfloor \log 100{,}000 \rfloor + 1 = 17$, but the missing-key test measured 16. That is because a key smaller than everything follows the leftmost path through the search, which is one step shorter than the deepest path.

None of these differences change which algorithm scales best. They show that the theory predicts growth very well but is not a precise predictor of real running time.

8. Recommendation for the Original Design

I recommend binary search on a sorted array.

Theoretical efficiency. Searches cost $\Theta(\log n)$, with a guaranteed worst case of $\lfloor \log n \rfloor + 1$ comparisons whatever the registration numbers look like. The one-time sort costs $\Theta(n \log n)$.

Measured performance. At 1,000,000 records an average search took 7.9 µs and 19.5 comparisons, about 7,000 times faster than linear search (55,103 µs). The sort took 4.5 s, which is repaid after about 82 searches.

Why not linear search? It has no setup cost, but it was already 20 times slower than binary search at 1,000 records and its cost grows in direct proportion to the database. It is only sensible for tiny datasets or a one-off lookup.

Why binary search over hashing, even though hashing was faster? Hashing did win on speed and on build time. I still choose binary search for the original problem for four reasons:

1. The original problem does not say how often searches happen, and without that, the guaranteed worst case matters more than the best average.
2. Binary search needs no extra data structure and uses only the sorted array.

3. A sorted array supports ordered queries, for example all students in a registration range, which a hash table cannot do.

4. The absolute difference (7.9 µs against 1.3 µs per search) is tiny until search volume becomes very high. If that happens, a hash index can be added cheaply, which is exactly what Section 9 does.

Trade-offs and limits. The sorted array is awkward to update (inserting a record means shifting records), so it suits data that changes rarely. The sort must be repeated or merged when records are added. It stays practical as the database grows, since each doubling adds just one comparison.

9. Day-4 Design Challenge

New conditions: 50,000 searches per hour on 1,000,000 records, with new records added only once per day.

A. Impact analysis

The load is now steady and heavy. 50,000 searches per hour is about 14 per second, or about 1.2 million per day if the system runs around the clock. Three original assumptions change:

1. **Preprocessing is no longer a large share of the cost.** One daily rebuild is now shared across about 1.2 million searches.

2. **Constant updates are no longer expected.** Inserting into a sorted array is $\Theta(n)$, which would hurt with continuous updates. With one batch a day, the structure can simply be rebuilt overnight.

3. **Small per-search differences now add up.** Saving even a few microseconds per search is multiplied by 1.2 million.

Search frequency matters because total cost is preprocessing plus q times the cost per search, and as q grows the second term dominates. The low update frequency matters because structures that are costly to build but cheap to query become attractive: they are built once and only read until the next day.

B. Evaluation of the original design

Binary search copes easily. At 7.9 µs per search and about 14 searches per second, searching uses roughly 110 µs of processing per second of real time. The daily sort (about 4.5 s) is trivial

when spread over 1.2 million searches, and the new records can be merged into the sorted array instead of sorting again.

Another approach does better, though. A hash index needs about 1.3 μs per search and a 2.3 s daily build, so its total daily cost is lower. Linear search, by comparison, would need about 55 ms per search. At 14 searches per second that is roughly 0.77 seconds of processing for every second of real time, which leaves almost no headroom for busy periods.

C. Decision: MODIFY

I keep the sorted array as the master copy and add a hash index from registration number to record position. The index is rebuilt once a day after the new records are merged in. Searches go through the index first. The sorted array stays for ordered queries and as a fallback with a guaranteed worst case.

What changes: one $\Theta(n)$ index build is added per day, and the per-search cost drops from $\Theta(\log n)$ to $\Theta(1)$ expected. The daily cost moves from $\Theta(n \log n + q \log n)$ to $\Theta(n + q)$.

Why not simply replace binary search with hashing? Keeping the sorted array costs almost nothing, since it already exists. It preserves range queries and a guaranteed $\Theta(\log n)$ fallback if the hash index ever misbehaves.

D. Evidence-based defence

The measured cost of one day (1,200,000 searches on 1,000,000 records):

Approach	**Preprocessing (s)**	**Searching (s)**	**Total per day (s)**
Linear	0	66,124.2	66,124.2
Binary (sorted daily)	4.5	9.5	14.0
Hash index	2.3	1.5	3.9

- **Linear search is ruled out.** It would use about 66,000 seconds of processing per day, which is around 77% of every second in a 24-hour day spent just on searching. It is about 4,700 times more expensive than binary search.
- **Between binary search and hashing the saving is real but modest in absolute terms.** The difference is about 10 seconds per day. Both handle 14 requests per second with

huge margin. Even so, hashing has the better prospects for growth, because its search cost does not rise with n while binary search's does.

- **Preprocessing is repaid almost immediately.** Break-even versus linear search is about 82 searches for the sorted array and 42 for the hash index. That is about 0.007% and 0.004% of one day's traffic.

The decision is therefore a measured one. It does not rest on binary search failing. It rests on the hash index giving lower cost and more headroom at a small price (one linear-time rebuild a day and some extra memory), while the sorted array keeps its useful guarantees.

10. Limitations

- **One run per dataset size.** Each time is an average over 100 queries, but I did not repeat whole runs, so I cannot quote error bars. The times would shift slightly between runs.
- **Python overhead.** Function calls, the comparison counter and list handling all add time. The overhead is the same for all three algorithms, so the comparison between them is fair, but absolute times would be much smaller in a compiled language.
- **Random keys.** Real registration numbers usually follow a pattern (year, department, serial number). Patterned keys are where hashing is most at risk, which is why a prime table size matters and why I would keep the sorted fallback.
- **Memory was not measured.** The hash index needs extra space proportional to n, and I did not quantify it.
- **Everything runs in memory.** A real student database would live on disk and would likely use an indexed structure such as a B-tree, which this assignment does not cover.
- **Half of the queries were for missing keys.** This was deliberate, to test failure cases, but it shifts the averages compared with a purely successful workload (see Section 7.1).

11. Conclusion

The experiments back up the theory. Linear search grows in step with the database and becomes unusable at a million records. Binary search needs only about 20 comparisons at that size, and hashing needs about one. Both pay a preprocessing cost, but that cost is repaid within tens of searches. The theory predicted the growth of every algorithm accurately, though real timings were affected by memory behaviour and Python overhead.

For the original problem, binary search on a sorted array is the safest balanced choice. When the system must handle 50,000 searches per hour with daily updates, adding a hash index on top of the sorted array gives the best overall result.

Appendix: Task Coverage

Task	**Where it is covered**
1. Algorithm design	Section 3
2. Basic operation	Section 4.1
3. Time-complexity derivation	Sections 4.2 and 4.3
4. Best, average, worst case	Section 4.4 (and 6.3 for the experiments)
5. Asymptotic analysis	Section 4.5
6. Recurrence formulation	Section 4.6
7. Recurrence solution	Section 4.7
8. Implementation and testing	Sections 5.1 to 5.3
9. Experimental performance	Section 6 and Figure 1
10. Theory versus experiment	Section 7
11. Final recommendation	Section 8
12. Day-4 challenge	Section 9